

BIG DATA INFRASTRUCTURE & ARCHITECTURE

Big data infrastructure

What kind of hardware do we need?

Scaling

Big data is... *big*

- It doesn't fit a single drive (anche se abbiamo un drive con Big data, non sarebbe comunque la soluzione migliore per quanto riguarda il Processing)
- It doesn't fit a single (typical) machine (non bastano multipli drive su un'unica macchina)



Year: 1990
Size: 1.3 GB
Speed: 4,4 MB/s

5 minutes



Year: 2014
Size: 1 TB
Speed: 100 MB/s

3 hours



Year: 2015
Size: 1 TB
Speed: 600 MB/s

30 minutes



Year: 2025
Size: 4 TB
Speed: 6 GB/s

11 minutes

(time to read the whole disk)

Scaling

Two ways to scale

- **Scale-up** (vertical scaling)
 - Adding more resources (processors, RAM, disks)
 - Upgrading machines (e.g., buying a more expensive and robust server, buying specialized hardware)
- **Scale-out** (horizontal scaling)
 - Adding more machines, creating clusters

lo si può fare fino ad un certo punto (i processori e le memorie hanno un limite)

alle macchine che già abbiamo

Soluzione preferita in Big Data

non ci sono limiti

Shared-memory architecture

Symmetric MultiProcessing

L'architettura a memoria condivisa (nota anche come Symmetric MultiProcessing o SMP) è il modello hardware tradizionale utilizzato nella maggior parte dei PC di uso comune.

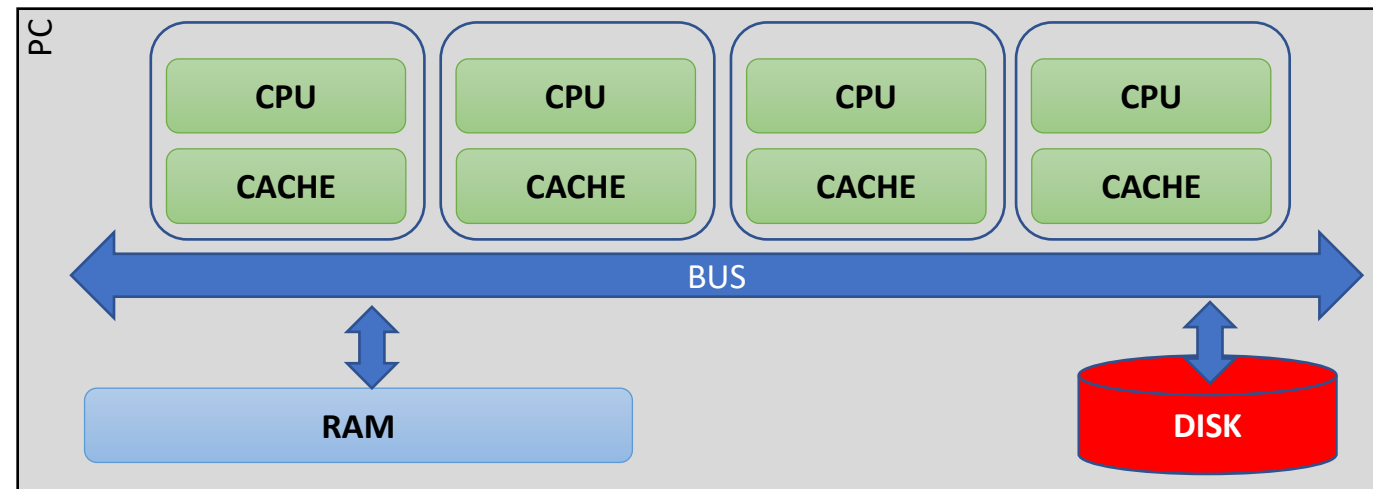
- Adopted by traditional notebooks and workstations
- Several processors share the same RAM, the same I/O bus and the same disk(s)



Limits

- Physical number of devices that can be mounted or plugged-in
- BUS bottleneck (cannot go over a certain speed)

oltre una certa soglia di velocità del bus, l'aggiunta di altri processori non porta alcun beneficio prestazionale perché la trasmissione dei dati diventa il fattore bloccante.



Una singola scheda madre ha dimensioni fisiche limitate e un numero finito di slot di espansione (socket per le CPU, slot per la RAM, connettori per i dischi). Questo pone un limite hardware rigido alla possibilità di aggiungere risorse (approccio scale-up): arrivati a un certo punto, non è più possibile inserire ulteriore memoria o altri processori nella singola macchina

Shared-disk architecture

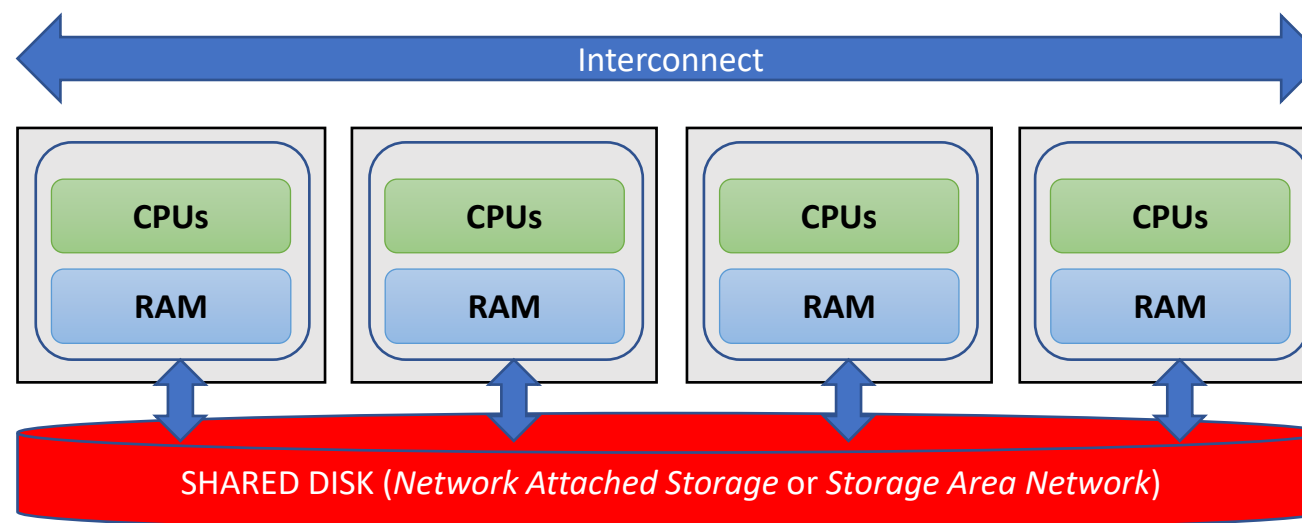
Più macchine o più unità di computazione separati dove ognuna ha la propria RAM ed un unico disco per ogni gruppo

A group of computing nodes, working together to solve a common goal

- Each (group of) computing unit has its own memory
- (Groups of) computing units communicate with the others through the interconnect

Limits

- Not designed for scaling to big data sizes (in terms of internal protocols and hardware)
- Concurrency capabilities are limited

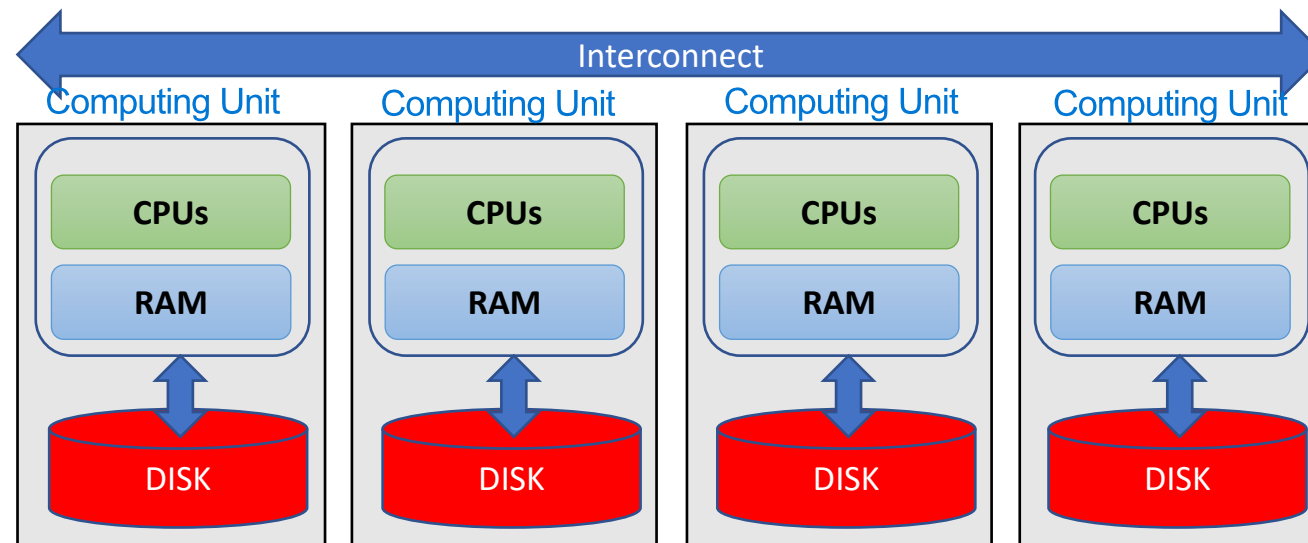


L'Interconnect (o rete di interconnessione) è l'infrastruttura di rete ad alta velocità che collega tra loro i singoli nodi di calcolo (server/macchine autonome) all'interno di un cluster o di un'architettura distribuita.

Shared-nothing architecture

A group of computing nodes, working together to solve a common goal

- Each (group of) computing unit has its own resources *Macchine indipendenti (hanno ognuno il proprio disco, ram e cpu)*
- (Groups of) computing units communicate with the others through the interconnect
- Scalability is virtually-unlimited *In questo caso, non abbiamo problemi di scalabilità in nessuno dei componenti (quando abbiamo bisogno di più risorse, basta semplicemente scalare).*



L'Interconnect (o rete di interconnessione) è l'infrastruttura di rete ad alta velocità che collega tra loro i singoli nodi di calcolo (server/macchine autonome) all'interno di un cluster o di un'architettura distribuita.

Il cluster adotta il modello shared-nothing, in cui non esistono dischi o memorie condivise tra i nodi. Ciascun nodo gestisce in autonomia le proprie risorse hardware e la propria porzione di dati, eliminando i colli di bottiglia centralizzati e permettendo un'elevata scalabilità orizzontale (scale-out).

Cluster

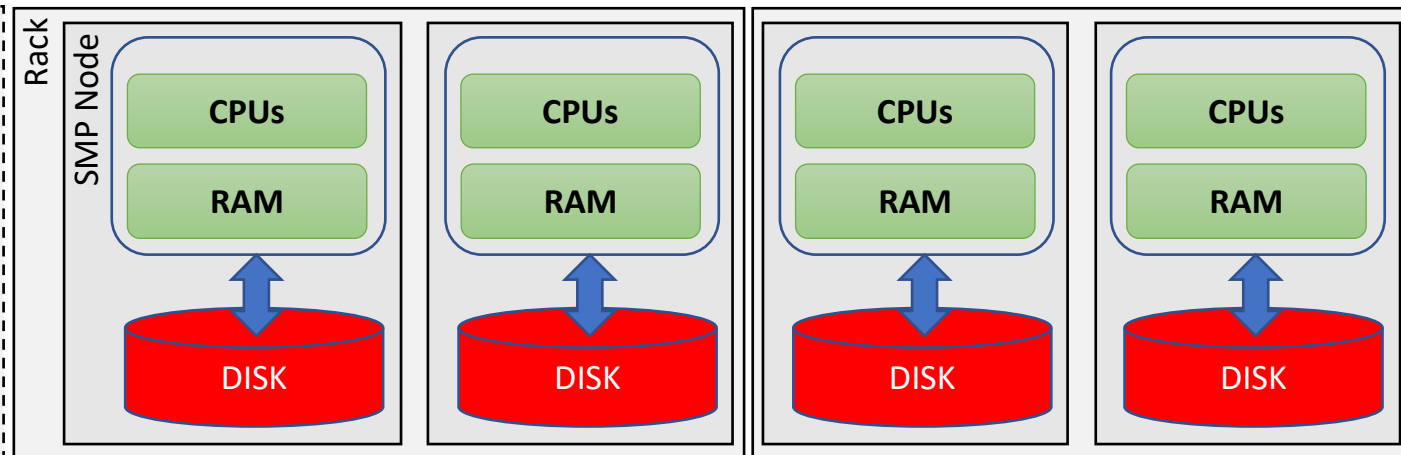
Un cluster è un insieme di computer autonomi (detti nodi) interconnessi tra loro che cooperano per eseguire compiti complessi, apparendo all'esterno come un'unica macchina coesa e integrata.

A cluster is a group of autonomous computers (*nodes*) that work together so that they can be viewed as a single system

- **Every node is a system on its own (SMP)** Ciascuna macchina del cluster è un computer indipendente basato su architettura Symmetric MultiProcessing (memoria condivisa con i propri processori, la propria RAM e il proprio bus/disco locale).
- **Typically connected to each other through fast LANs (Ethernet or InfiniBand)** → rete ad altissima velocità e bassa latenza.
- **Typically adopts a shared-nothing architecture** (puoi implementare il cluster anche con componenti in shared-disk architecture, ma è molto raro)
- **Stored in racks → Scalability comes at the price of network speed**

I nodi vengono alloggiati fisicamente in strutture a scaffale chiamate rack. Aggiungere nodi e rack aumenta la capacità computazionale e di memoria, ma introduce un compromesso sulla velocità: trasferire dati tra nodi nello stesso rack (intra-rack) è molto più veloce che trasferirli tra nodi situati in rack differenti (inter-rack). La rete diventa quindi il principale collo di bottiglia potenziale nell'aumentare la scala del cluster.

Ethernet/InfiniBand



All'interno di un singolo rack, ogni nodo è collegato direttamente allo switch locale (Top-of-Rack) con una linea ad alta velocità dedicata (ad esempio 10 Gbps). Se un rack contiene 32 nodi, la capacità di traffico interna è elevata (320 Gbps totali). Tuttavia, il cavo che collega lo switch del rack ai livelli di rete superiori (Aggregation Switch ed altri) ha una capacità molto inferiore alla somma dei singoli nodi (ad esempio 40 Gbps). Di conseguenza, quando i dati devono uscire dal rack, i 32 nodi devono contendersi quei 40 Gbps, riducendo drasticamente la larghezza di banda effettiva per ciascun nodo. Man mano che si aggiungono livelli di rete (switch di aggregazione etc etc), lo stesso canale di comunicazione deve trasportare contemporaneamente i dati di un numero sempre maggiore di rack e nodi. Più rack cercano di comunicare all'esterno nello stesso momento, maggiore sarà la saturazione della larghezza di banda comune, riducendo la quantità di dati che ogni singolo nodo riesce a inviare al secondo.

Cluster

Compute nodes are stored on racks

- 8–64 compute nodes on a rack
- There can be many racks of compute nodes
- Intra-rack bandwidth >> inter-rack bandwidth
- Increasing networking layers decreases the overall bandwidth

Disponi i rack in una gerarchia ad albero

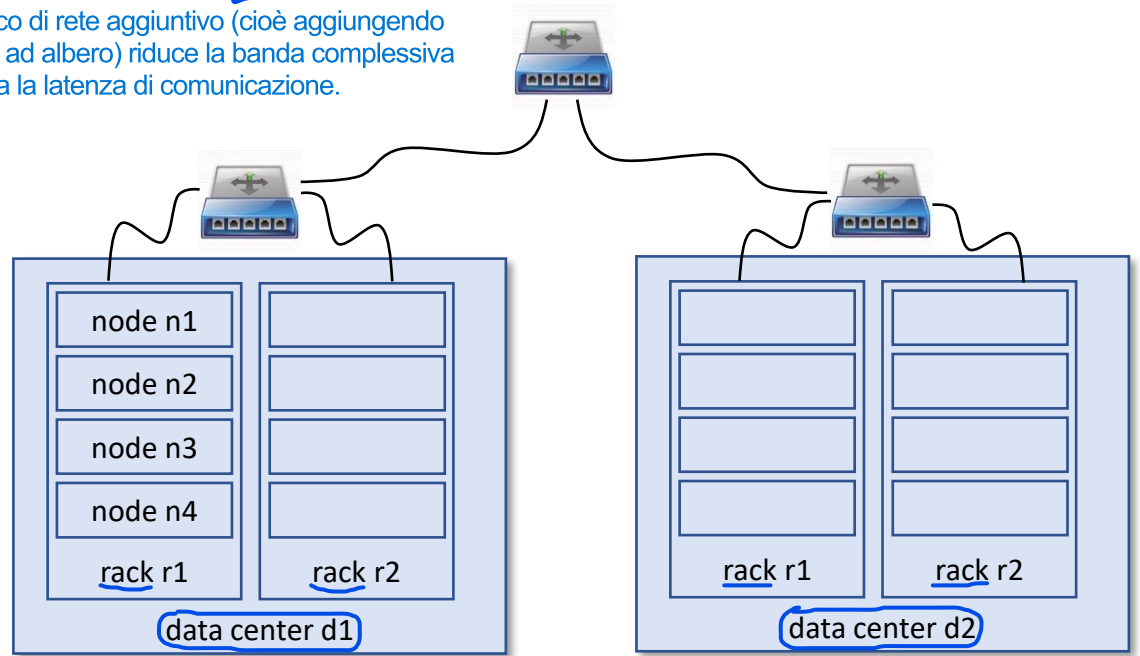
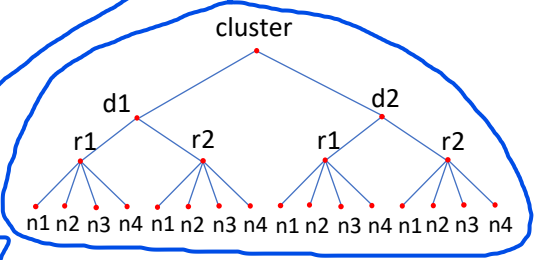
Ogni livello gerarchico di rete aggiuntivo (cioè aggiungendo livelli nella gerarchia ad albero) riduce la banda complessiva sfruttabile e aumenta la latenza di comunicazione.

Puoi fare Horizontal scaling quanto vuoi, però devi considerare il problema della comunicazione tra rack e tra data center (larghezza di banda e latenza)

Un cluster di grandi dimensioni è composto da molti rack distinti distribuiti nel data center

- Intra-rack (interna al rack): La comunicazione tra nodi dello stesso rack avviene tramite uno switch locale (Top-of-Rack), garantendo la massima velocità di trasferimento dati e latenze minime.
- Inter-rack (tra rack diversi): La banda disponibile per la comunicazione tra nodi appartenenti a rack differenti è nettamente inferiore (Intra-rack >> inter-rack).

Ogni rack ha un proprio Top-of-rack switch; i rack di uno stesso data center comunicano con Aggregation switch



Cluster

Una engineered appliance è una soluzione in cui hardware e software sono progettati, testati e ottimizzati per lavorare insieme come un unico blocco integrato. Nel nostro caso, si richiede la costruzione di un cluster pre-configurato con il Software richiesto, Hardware specializzato per il lavoro richiesto, caratteristiche richieste (high availability, reliability etc etc) ed infrastruttura

Clusters can be built/acquired in multiple ways (1/2)

■ Engineered appliance

- A specific vendor (e.g., Teradata, IBM) ^{vende o affitta} sells/rents a specialized infrastructure
 - Teradata IntelliFlex/VantageCore
 - IBM Netezza Performance Server
 - Microsoft APS/PDW
 - Oracle Exadata

High cost ■ Very high performances and ^{assenza di problemi di configurazione iniziale} no setup issue **BUT** ^{vincolo totale al fornitore (per ogni modifica, ho bisogno di chiedere al fornitore)} suffers vendor lock-in and limited scalability ^(la soluzione viene venduta con un gruppo di rack che può essere scalata fino ad un numero limitato di rack)

■ On-premise

- Buy *commodity hardware* and build your own cluster ^{nei propri data center}
- Maximum flexibility **BUT** ^(spazio, alimentazione, raffreddamento) suffers high setup and maintenance costs, logistic issues, inefficiency risks

^{You need to have Technical knowledge and skills about}

Cluster

Clusters can be built/acquired in multiple ways (2/2)

- **IaaS (Infrastructure as a Service)**
 - Rent single ^{macchine virtuali o server} machines from a cloud provider
 - Removes logistic costs and adds elasticity **BUT** keeps configuration costs, renting is costly, and there are still inefficiency risks
- **PaaS (Platform as a Service)**
 - Rent ^{pre-configurati e gestiti direttamente} managed clusters from a cloud provider
 - Removes configuration costs **BUT** adds an extra layer of costs ^{rispetto allo IaaS}
- **SaaS (Software as a Service)**
 - Use products/services offered by a cloud provider
 - The cluster is there – you just don't see it
 - ↳ L'infrastruttura del cluster è del tutto trasparente per l'utente ("il cluster c'è, ma non si vede"), che interagisce solo con l'interfaccia o le API del servizio senza gestire alcun parametro hardware o di sistema.

Commodity hardware

(example: buy from HP, Asus, etc etc)

If they have the same capacity, speed, they are the same

Clusters are usually made with **commodity hardware**

- Except with engineered appliances

A *commodity* is something (a ^{bene}good, a product) that

- Can be bought from a wide range of vendors..
- ..at more or less the same price..
- ..without being able of distinguishing where it comes from

Some examples: gold, electricity

Pros

- **No vendor lock-in** È possibile acquistare e integrare nodi da produttori differenti (es. Dell, HP, Lenovo) senza dipendere da un unico fornitore per aggiornamenti o parti di ricambio.
- **Interchangeability/interconnectivity** L'adozione di standard industriali condivisi rende semplice la sostituzione dei componenti guasti e la scalabilità progressiva della rete.

Cons

- **You need to deal with failures**
- **Commodity ≠ low-end! Cheap components with high failure rate can be a false economy**

Poiché i componenti standard non hanno la ridondanza hardware estrema dei grandi mainframe, i guasti ai singoli nodi o dischi sono eventi quotidiani e inevitabili. L'infrastruttura software per i Big Data (come HDFS o Spark) deve essere quindi progettata per gestire i guasti a livello applicativo (es. tramite replicazione dei dati).

"Commodity" non significa hardware economico di scarsa qualità o di fascia bassa. Utilizzare componenti scadenti per risparmiare sui costi iniziali genera un tasso di guasto elevatissimo, trasformandosi in una "falsa economia" a causa dell'aumento dei costi di manutenzione, dei tempi di inattività (downtime) e delle perdite di prestazioni.

Cluster

Indipendentemente da come costruisci il Cluster,

è un'opzione allettante per
Having a single large cluster is tantalizing to many organizations

Poiché evita la creazione di "silos di dati" (ovvero dati isolati in vari reparti) e garantisce una governance e un controllo degli accessi più semplici.

- **No data silos, simpler governance**

Tuttavia

Multiple clusters are inevitable within medium-large enterprise settings

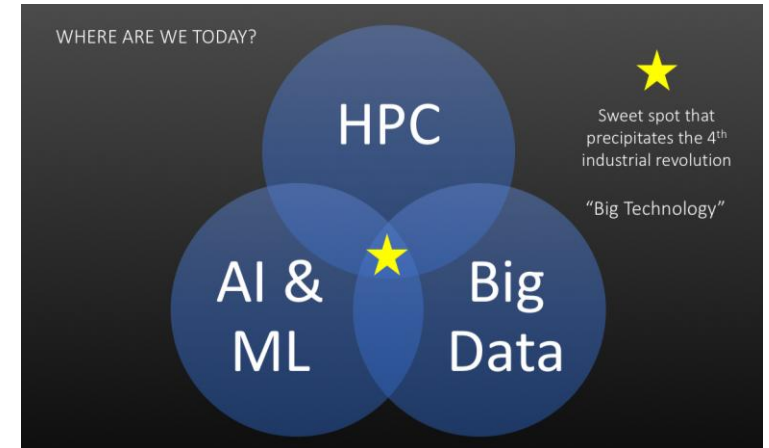
- **Resiliency** Resilienza Un singolo cluster si trova solitamente all'interno di un'unica posizione geografica o data center, costituendo un Single Point of Failure (SPOF). Un guasto alla rete elettrica, un problema di connettività o un evento catastrofico locale arresterebbe l'intera operatività aziendale. Distribuire le operazioni su più cluster geografici garantisce la continuità operativa.
 - **Every cluster sits within a single point of failure due to geography** è esposto ad un unico
- **Software development** Separare i cluster di produzione da quelli dedicati a test, integrazione e sviluppo evita che l'esecuzione di test intensivi, configurazioni errate o bug nel codice impattino le applicazioni critiche in ambiente di produzione.
 - **Mitigate the risk of impacting critical production environments by isolating configuration, integration, or evolution testing and deployment**
- **Workload isolation** Cluster distinti evitano contese di risorse e permettono di configurare hardware dedicato al tipo di carico (es. nodi ad alta densità di GPU per la ML rispetto a nodi con molta RAM per query in-memory).
 - **Hardware resources tuned for specific workloads, less resource contention**
- **Legal separation** (example: in EU, we have GDPR; in USA, we have other restrictions)
- **Separate storage from compute to scale differently – especially in cloud implementations**

→ Questo disaccoppiamento permette di scalare lo spazio di archiviazione in modo indipendente dalla potenza di calcolo, accendendo o spegnendo cluster di calcolo solo quando strettamente necessario.

Cluster

What about High-Performance Computing?

- HPC generally refers to massively parallel system specifically devoted to solve **computationally-intensive tasks**
 - HPC main applications are scientific simulations, weather forecasts, 3D modeling, etc.
 - Usage of GPUs is increasing
- Big Data systems are mostly **data-intensive** (problem related more on size and complexity of data than on computation)
- The line separating Big Data and HPC clusters is very much blurred



Examples @PoliTo 2 clusters (example to see the difference between HPC and Big data clusters)

- Big Data computing cluster
 - 1400+ physical cores, 15 TB RAM, 8 PB storage, 6 nVidia V100 GPUs
- HPC computing cluster
 - 2900+ physical cores, 27 TB RAM, 256 TB storage, 32 nVidia V100 GPUs, 100Gb/s Infiniband EDR

Reference Big Data Architectures

Software to communicate and coordinate between Machines computation and storage

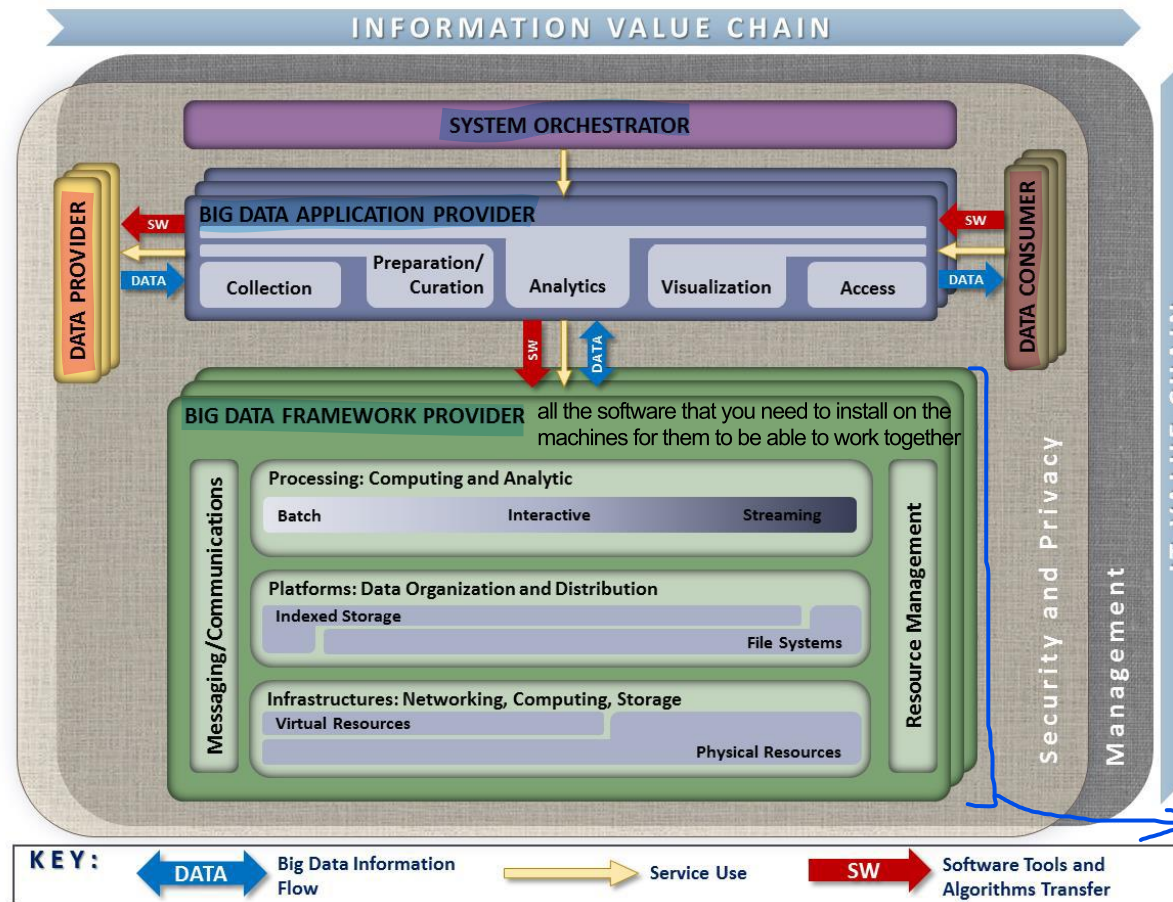
What kind of software do we need?

NIST's reference architecture

Ci dice quali sono ad alto livello i componenti principali delle architetture Big Data

- Data Provider: La sorgente che introduce i dati grezzi nel sistema (sensori, utenti, log o sistemi esterni), occupandosi di raccolta iniziale, trasformazione preliminare, generazione dei metadati e gestione delle autorizzazioni di accesso.
- Big Data Application Provider: Il livello in cui risiede la logica applicativa del sistema (applicazione che usa i framework per essere costruita/supportata), strutturato in cinque attività consequenziali:
 - > Collection: Acquisizione dei flussi dati in ingresso.
 - > Preprocessing: Pulizia, normalizzazione, scrubbing e arricchimento dei dati.
 - > Analytics: Esecuzione di algoritmi analitici, aggregazioni e modelli di Machine Learning.
 - > Visualization: Generazione di report e dashboard grafiche per la fruizione dei risultati.
 - > Access: Esposizione di interfacce e API per l'interrogazione dei dati.
- Big Data Framework Provider: Il livello infrastrutturale e di piattaforma che eroga le risorse necessarie all'applicazione, suddiviso in:
 - > Infrastructures: Risorse fisiche e virtuali (OS, Docker, reti, nodi di calcolo, storage and altre cose di basso livello).
 - > Platforms: Gestione dell'organizzazione dei dati (File Systems distribuiti come HDFS, tabelle analitiche come Iceberg e Indexed Storage NoSQL).
 - > Processing: Motori di esecuzione suddivisi in modelli Batch, Interactive e Streaming.
 - > Messaging/Communications & Resource Management: Moduli per la trasmissione dei messaggi (es. Kafka) e la gestione dinamica delle risorse del cluster (es. YARN, Kubernetes).
- System Orchestrator: Il coordinatore centrale (software o persona) che gestisce e automatizza l'esecuzione dei workflow applicativi, allocando le risorse (richieste al Resource management, non sempre date quanto richieste) ed effettuando il monitoraggio dei processi di calcolo (gestisce e coordina tutto il sistema).
- Data Consumer: L'entità finale (utente, applicazione aziendale o sistema esterno) che consuma i dati raffinati messi a disposizione dall'Application Provider.

<https://bigdatawg.nist.gov/uploadfiles/NIST.SP.1500-6r1.pdf>

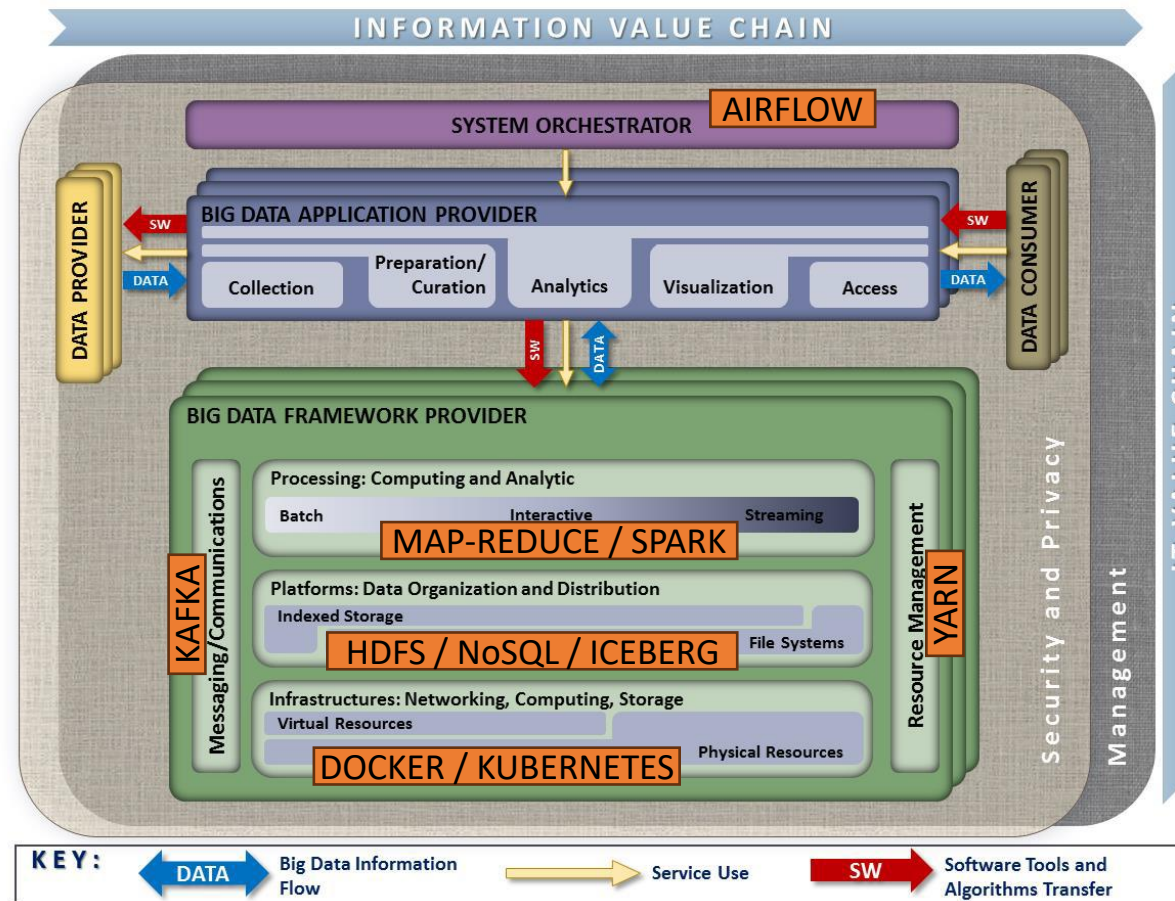


Il Big Data Framework Provider rappresenta l'insieme di strumenti software e piattaforme che permettono all'infrastruttura di macchine distribuite di comunicare, coordinarsi e operare come un unico sistema integrato. Questo livello astrae l'hardware sottostante e fornisce i servizi fondamentali di calcolo, archiviazione e gestione delle risorse necessari per l'esecuzione delle applicazioni Big Data. Il livello si articola in quattro componenti principali:

- Infrastructure Frameworks: Gestisce le risorse fisiche o virtuali di calcolo, rete e archiviazione (es. sistemi operativi, Docker, Kubernetes).
- Data Platform Frameworks: Gestisce la distribuzione, la strutturazione e l'organizzazione dei dati sui vari nodi (es. file system distribuiti come HDFS, database NoSQL, formati di tabella come Apache Iceberg).
- Processing Frameworks: Mette a disposizione i motori computazionali per analisi/operazioni di tipo batch, streaming o interattivo (es. Apache Spark, MapReduce per elaborazioni batch, interattive o streaming).
- Resource Management & Messaging (software ortogonale rispetto agli altri 3 livelli): Gestisce l'assegnazione dinamica delle risorse alle applicazioni che gireranno sul cluster e lo scambio rapido di messaggi/eventi tra applicazioni/software che girano su macchine diverse del cluster (es. YARN, Apache Kafka).

NIST's reference architecture

Most used tools
for each category



<https://bigdatawg.nist.gov/uploadfiles/NIST.SP.1500-6r1.pdf>

NIST's reference architecture

Il provisioning è il processo di allocazione, configurazione e messa a disposizione di risorse IT (hardware, macchine virtuali, storage, reti, sistemi operativi e piattaforme software) affinché siano pronte all'uso per utenti o applicazioni. In ambito Big Data, il provisioning descrive la modalità con cui il cluster e il relativo stack software vengono creati e preparati all'elaborazione dei dati:

- Provisioning Manuale (Bare-metal): Configurazione diretta dei server fisici, con installazione e parametrizzazione manuale del sistema operativo, delle connessioni di rete e dei framework di elaborazione (es. HDFS, Spark, YARN).
- Provisioning IaaS (Infrastructure as Service): Allocazione di macchine virtuali e volumi di storage da un provider Cloud tramite script automatizzati o interfacce API.
- Provisioning Gestito (PaaS/Cloud): Creazione automatizzata di cluster pre-configurati (es. AWS EMR, Google Cloud Dataproc), dove il Cloud Provider istanzia i nodi e configura lo stack software in autonomia.

In short

- **Big Data Framework Provider**
 - The ecosystem of tools (+ the hardware) that you need
 - Provisioning can range from manual (on bare metal) to fully-managed (on cloud)
 - You need to know how the tools work to make your own stack
- **Big Data Application Provider**
 - The programs ^{che} ~~the~~ must be written to process the data, from collection to visualization
- **System Orchestrator**
 - The puppet master
 - Decides which programs should be run, when, and how
 - Could be manually done or automated



NIST's reference architecture

Big Data Framework Provider

- **Provides general resources or services to be used by the Big Data Application Provider in the creation of the specific application**
 - Handles the communication with the Big Data Application Provider and the assignment of physical resources to the respective activities
- **Consists of Infrastructure frameworks..**
 - Support the underlying **computing, storage, and networking** functions required to implement the overall system
 - Associated with physical or virtual infrastructure resources
- **.. Data Platform frameworks..**
 - Manage the **organization and distribution of data** (file System, databases, etc.)
- **.. and Processing frameworks**
 - How data will be processed in support of Big Data applications (e.g., **batch/streaming framework**)

NIST's reference architecture

Data Provider

- **Introduces new data or information feeds into the Big Data system**
- It can be anything from a sensor to a human or another Big Data system
- Includes the following activities:
 - Collecting and persisting the data
 - Providing transformation functions for scrubbing sensitive information
 - Creating the metadata describing the data source, usage policies/access rights, etc.
 - Enforcing access rights on data access and establishing (in)formal contracts for data access authorizations
 - Making the data accessible through suitable programmable push or pull interfaces and mechanisms
 - Publishing the availability of the information and the means to access it

Data Consumer

- **Uses the interfaces or services provided by the Big Data Application Provider to get access to the information of interest**
- It can be an actual end user or another Big Data system

NIST's reference architecture

Big Data Application Provider

- **Executes a specific set of operations (pipeline/workflow)**
 - Meet the requirements established by the System Orchestrator
 - Meet security and privacy requirements
- **Includes** the following activities:
 - Collection, Preparation, Analytics, Visualization, Access
- **Each activity**
 - Is specific to the application
 - Can be implemented by independent stakeholders and deployed as a stand-alone service
- There may be multiple and differing instances of each activity, or a single program may perform multiple activities
- Each of the functions can run on a separate Big Data Framework Provider or all can use a common Big Data Framework Provider

NIST's reference architecture

System Orchestrator

Unisce e collega tra loro le singole attività dell'applicazione (raccolta, preparazione, analisi, visualizzazione e accesso) facendole funzionare come una pipeline sequenziale e coesa, in grado di trasformare i dati grezzi in informazioni utili.

- **Integrate the required application activities into an operational vertical system**
- **Configure and manage the other components of the Big Data architecture to implement one or more workloads**
- **Monitor workloads and system to verify the meeting of quality requirements**
- **Elastically assign and provision additional physical or virtual resources**
- **Performed by human and/or software components**

Interagisce direttamente con gli altri livelli dell'architettura (come il Big Data Framework Provider) impostando i parametri e coordinando i servizi affinché siano pronti a eseguire uno o più job di calcolo specifici.

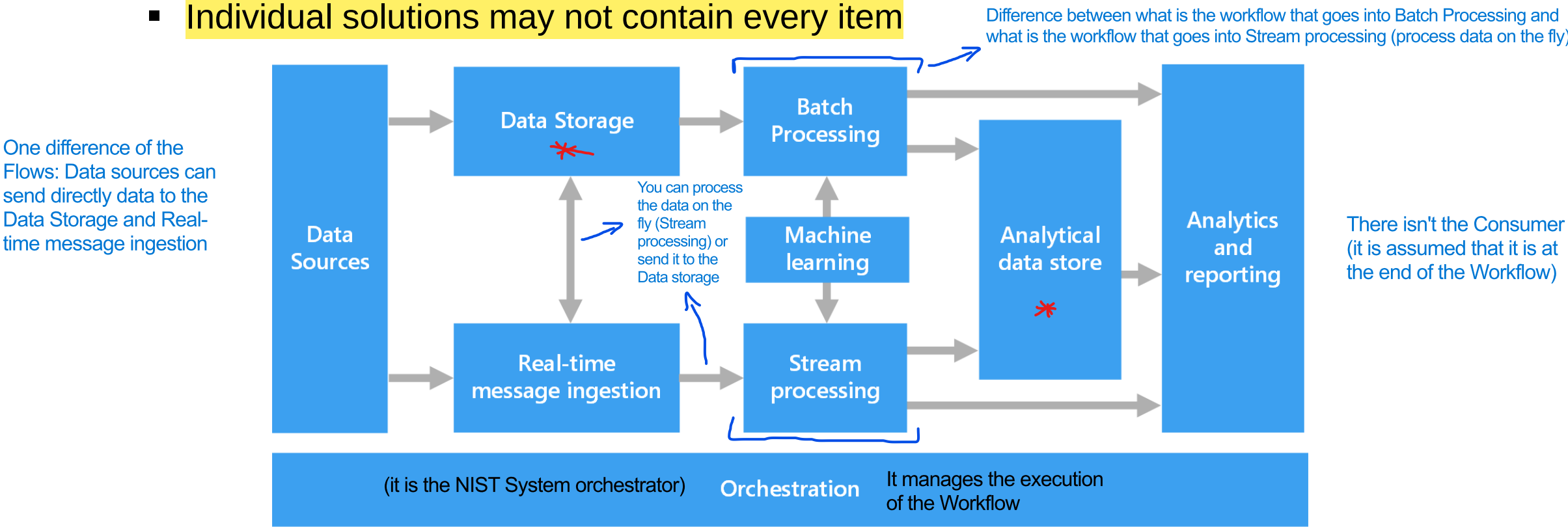
Gestisce la scalabilità dinamica del sistema; quando il volume dei dati aumenta o un job richiede più potenza di calcolo, l'orchestratore provvede ad aggiungere, configurare e assegnare risorse aggiuntive (come nodi o container) per poi rilasciarle a fine elaborazione.

Another reference Big Data Architecture from Microsoft. L'architettura di riferimento dei Big Data proposta da Microsoft si focalizza sul flusso logico dei dati e sulle operazioni che trasformano i dati grezzi provenienti dalle sorgenti in informazioni fruibili dai consumatori finali. Mentre il modello NIST definisce i ruoli e gli attori dell'infrastruttura, il modello Microsoft organizza il sistema in componenti logiche interconnesse, dove non tutte devono necessariamente essere presenti in ogni singola implementazione aziendale.

Components of a big data architecture

Logical components that fit into a big data architecture

- Individual solutions may not contain every item



- * Two different storage services (not necessarily two different tools: distinction between raw data and results of analysis)

Components of a big data architecture

Data sources I punti di origine dell'informazione, suddivisi in archivi strutturati (database), file statici (es. file log, CSV, Parquet) e sorgenti in tempo reale (es. sensori IoT, clickstream)

- Data stores, static files, real-time sources

Data storage

- Distributed file store (data lake) or database per contenere grandi volumi di dati grezzi

Batch processing (eseguiti a intervalli programmati)

- Long-running batch jobs to filter, aggregate, and prepare the data for analysis

Analytical data store Database strutturati ottimizzati per organizzare i dati elaborati in un formato facilmente e rapidamente interrogabile dagli strumenti di analisi.

- Serve the processed data in a structured format that can be queried using analytical tools

Analysis and reporting Il livello di interfaccia finale con l'utente

- Traditional OLAP and BI tools, interactive exploration, analytical notebooks

Components of a big data architecture

Real-time message ingestion

Componente di memorizzazione temporanea che cattura flussi continui di messaggi ad alta velocità, agendo come buffer scalabile; garantisce semantiche di coda affidabili (message queueing) e tolleranza ai guasti senza perdere eventi.

- Capture and store real-time messages for stream processing
- Act as a buffer for messages
- Support scale-out processing, reliable delivery, message queueing semantics

Stream processing

- Filter, aggregate, and prepare the data for analysis
- Processed stream data is then written to an output sink

Orchestration

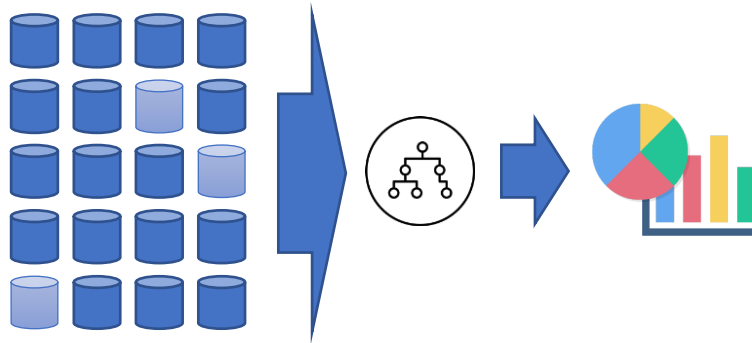
Il modulo di automazione orizzontale che definisce ed esegue i workflow operativi ripetitivi; gestisce le dipendenze, lo spostamento dei dati tra storage e sink, l'aggiornamento dei datastore analitici e il caricamento delle dashboard.

- Automation of repeated data processing operations, encapsulated in workflows
- E.g., transforming source data, moving data between multiple sources and sinks, loading into an analytical data store, pushing results to a dashboard

Two main analytical applications

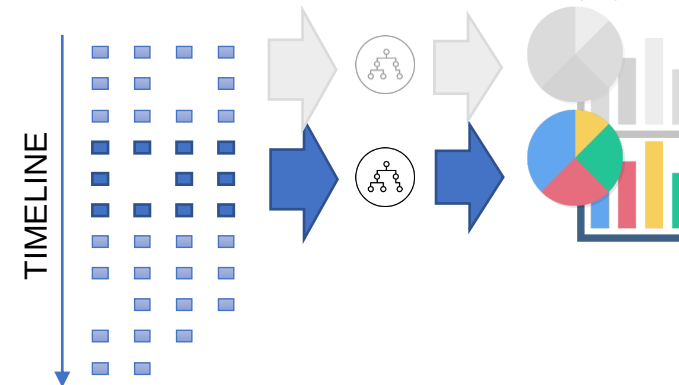
Batch analyses

- Analytical algorithms are launched over large amounts of stored data (at rest)
 - Typical analytical scenario
 - Complex algorithms may take minutes/hours
 - Run whenever needed (*on-demand*)



Stream analysis

- Analytical algorithms are continuously running (non-stop) over potentially infinite data as soon as it is collected
 - Results in near real-time
 - Useful for monitoring purposes
 - One-pass computation take the data, process it and discard them (they will never be process again from the algo)



Can we do both on the same data?

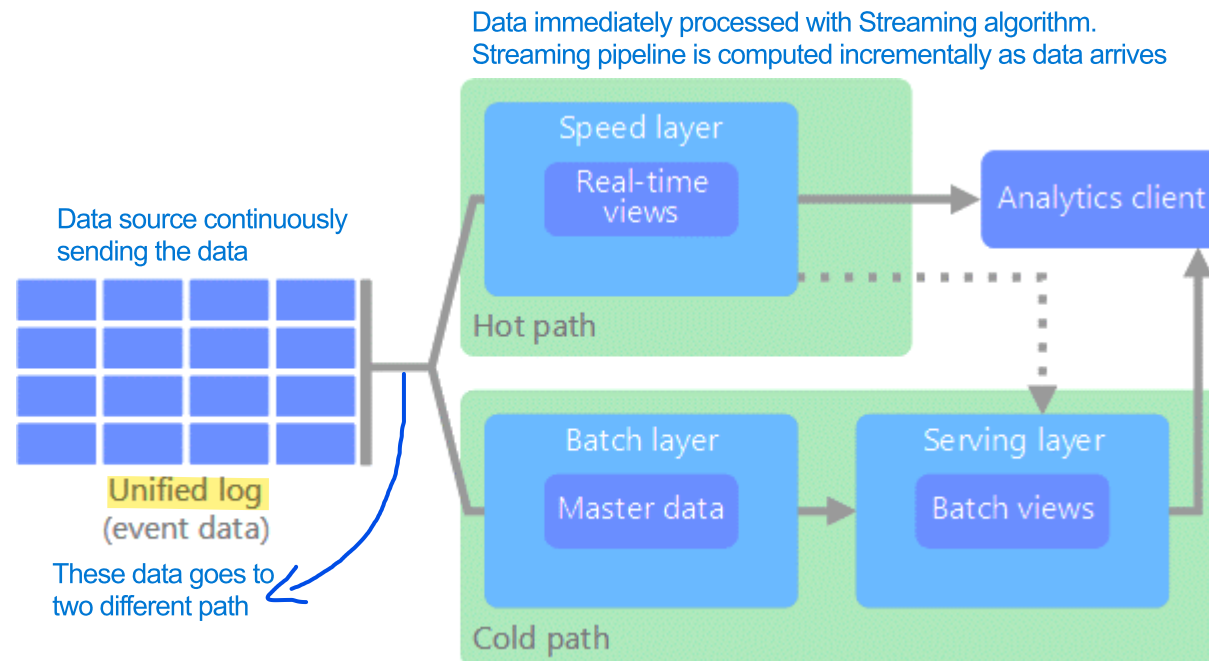
Yes. Le architetture Lambda e Kappa sono due tipi di architetture nati per gestire e integrare i due diversi tipi di elaborazione dei dati nei Big Data: l'analisi batch (su grandi volumi di dati a riposo, accurata ma lenta) e l'analisi stream (su flussi continui di dati in movimento, rapida ma focalizzata sul tempo reale).

Il nome lambda perché la lettera greca riporta alla mente i due path che si prendono dalla sorgente dei dati

Lambda architecture

All data coming into the system goes through two paths

- **Hot path:** ^{per dati in tempo reale, ma} ~~for timely~~, yet potentially **less accurate data** in real time
- **Cold path:** ^{per dati meno recenti} ~~for less timely~~ but more accurate data



Data immediately processed with Streaming algorithm.
Streaming pipeline is computed incrementally as data arrives

Data stored and in some point Batch algorithm are computed on the data stored.
Batch pipeline is computed on-demand (you process every data from the start)

L'architettura Lambda divide il flusso di dati in ingresso (Unified Log) sdoppiandolo su due percorsi distinti (Hot Path e Cold Path):

- Cold Path (Batch Layer + Serving Layer): I dati grezzi accumulati nello storage primario (Master Data) vengono elaborati periodicamente da job batch ad alta intensità. Genera Batch Views estremamente accurate e storicamente complete, con tempi di aggiornamento di ore o giorni.
- Hot Path (Speed Layer): Elabora immediatamente i dati appena arrivano, senza attendere il completamento del ciclo batch. Genera Real-time Views per fornire una stima aggiornata in tempo reale, accettando un potenziale margine di approssimazione o la mancanza di elaborazioni complesse sul pregresso.

Analytics Client: Unisce le Batch Views (storiche ed esatte) e le Real-time Views (recenti e veloci) per mostrare all'utente finale un quadro informativo completo.

Warren, J. and Marz, N., 2015. *Big Data: Principles and best practices of scalable realtime data systems*. Simon and Schuster.

Il nome Kappa perché, essendo Kappa un'architettura più semplice di Lambda, la lettera Kappa è prima della lettera Lambda

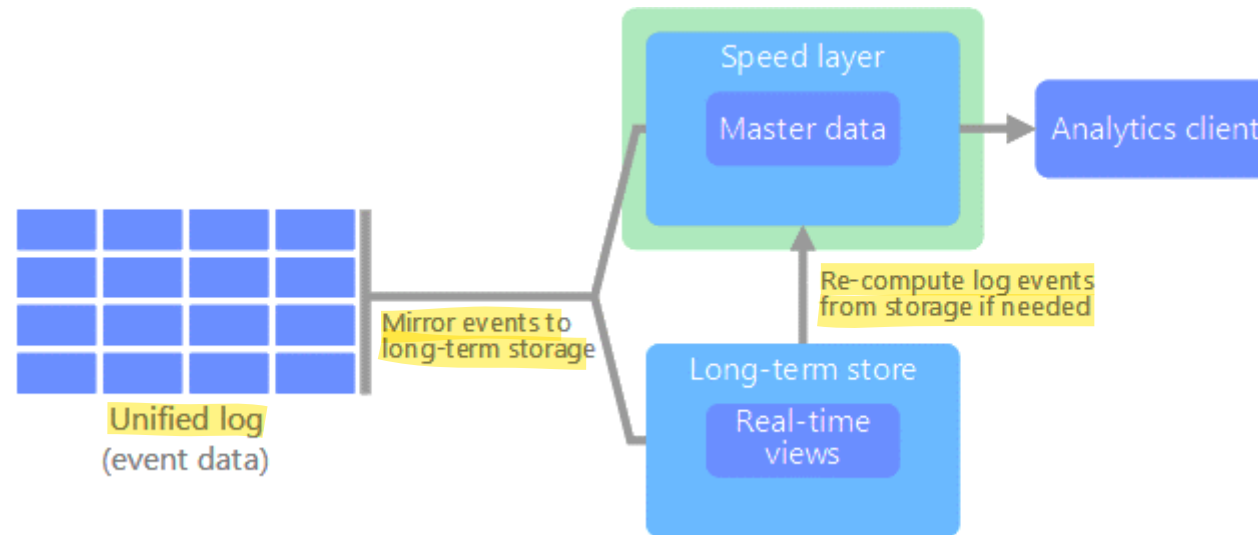
Kappa architecture

L'architettura Kappa semplifica il sistema eliminando completamente il Batch Layer: tutto il flusso di dati attraversa un'unica pipeline basata sullo stream processing (Speed Layer).

- Elaborazione continua: L'applicazione esegue l'analisi sui dati in transito generando direttamente le Real-time Views per gli utenti finali.
- Gestione del ricalcolo e dello storico: Per effettuare elaborazioni sul passato o ricalcolare i dati a seguito di una modifica del codice (operazione tradizionalmente svolta dal batch layer), l'architettura Kappa sfrutta uno storage immutabile a lungo termine e si limita a ri-eseguire (replay) lo stream di eventi attraverso il motore di streaming.

All data flows through a single path, using a stream processing system

- Similar to a lambda architecture's speed layer, all event processing is performed on the input stream and persisted as a real-time view
- To re-compute the entire data set (equivalent to what the batch layer does in lambda), simply replay the stream



We have only one pipeline and switch Data store if you want to run some Batch algorithm

Lin, J., 2017. *The lambda and the kappa*. IEEE Internet Computing, 21(5), pp.60-66.

"strict superset": un sistema di streaming moderno può fare tutto ciò che fa un sistema batch, più una serie di operazioni avanzate in più che il batch non è fisicamente in grado di compiere.

Lambda vs Kappa

I moderni motori di streaming (come Apache Flink) hanno introdotto meccanismi avanzati di checkpointing (fotografie periodiche e istantanee dello stato della memoria):

- Elaborazione Exactly-once: Se un nodo crasha durante il flusso, il sistema ripristina lo stato dall'ultimo checkpoint e rielabora solo gli eventi mancanti. L'impatto di ogni singolo evento sul risultato finale viene registrato esattamente una sola volta, azzerando sia le perdite che i doppi conteggi.
- Consistenza Forte (Strong consistency): Tutti i componenti del sistema vedono esattamente lo stesso stato dei dati nello stesso momento, senza disallineamenti.

Grazie a queste garanzie, il totale calcolato "al volo" dallo streaming è matematicamente identico e accurato a quello calcolato da un job batch su dati a riposo. Questo rende superflua la presenza di un layer batch separato per correggere gli errori.

Lambda used to be the easiest solution if we want to do both analysis

- But it needs the **parallel development and maintenance of two parallel pipelines**
- Same goal, different languages, different frameworks for different path

With Kappa, modeling everything under the streaming paradigm is **not** trivial

- **A streaming engine can handle a bounded dataset** Un motore di streaming può elaborare un dataset storico finito trattandolo semplicemente come un flusso che ha un inizio e una fine.
- Well-designed streaming systems ~~provide a strict superset of batch functionality~~ costituiscono un superset rigoroso delle funzionalità batch.
 - Introduce correctness (exactly-once semantics, strong consistency) and the result of a streaming job is the same of batch job Garanzia di correttezza: Con l'introduzione di semantiche avanzate come l'elaborazione exactly-once (ciascun evento viene elaborato esattamente una volta) e la consistenza forte (strong consistency), il risultato prodotto da un job in streaming è accurato e identico a quello di un job batch.
- Advanced tools provide a single framework and unified APIs for writing and executing both batch and streaming jobs
 - Google Cloud Dataflow
 - Apache Flink

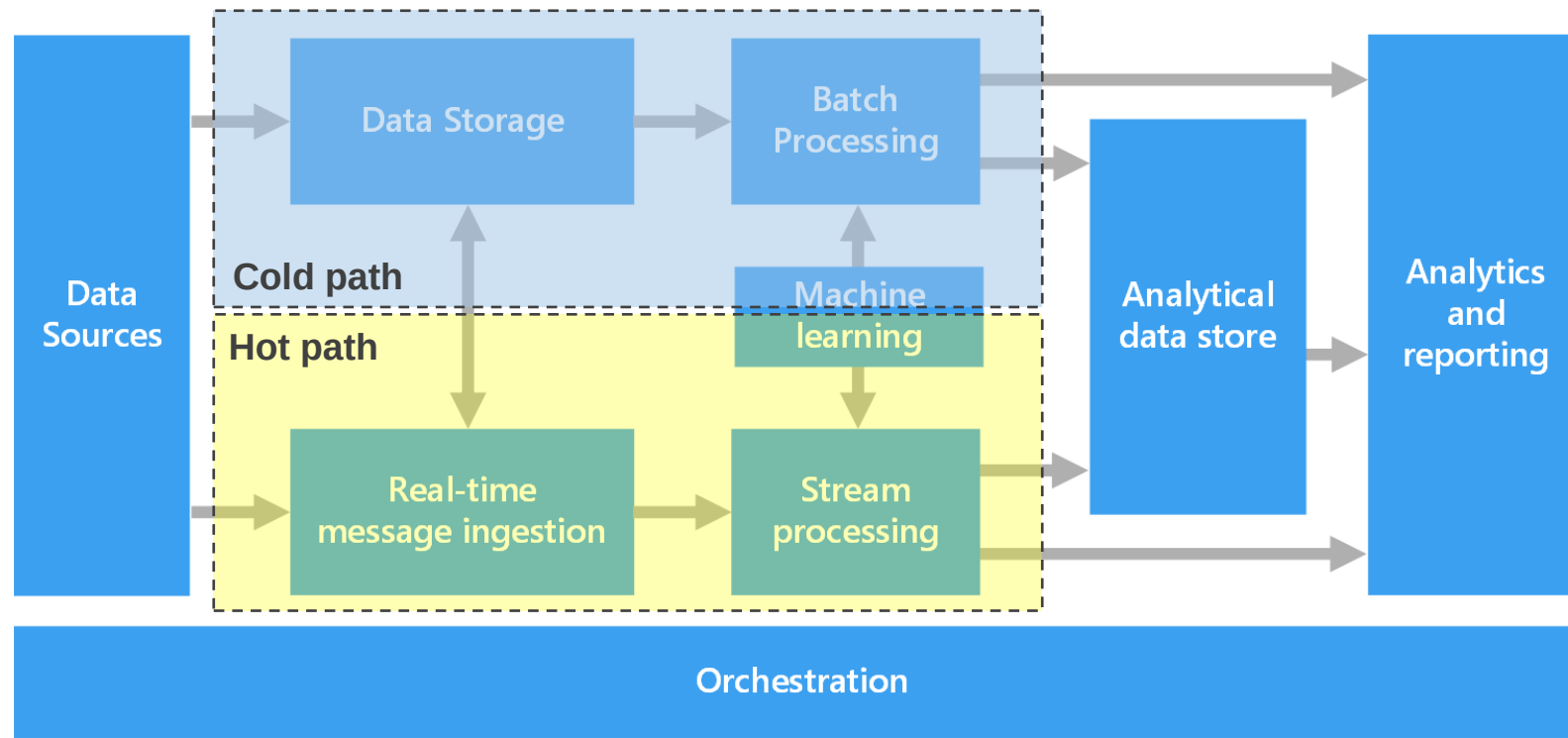
▪ *"Broad maturation of streaming systems combined with robust frameworks for unbounded data processing will in time allow for the relegation of the Lambda Architecture to the antiquity of big data history where it belongs"*

Il Declino dell'Architettura Lambda: La maturazione dei motori di streaming e dei framework per dati illimitati (unbounded data) sta gradualmente rendendo l'architettura Lambda un retaggio del passato, sostituita dalla semplicità architetturale del modello Kappa.

Streaming Systems. *The What, Where, When, and How of Large Scale Data Processing*. Tyler Akidau, Slava Chernyak, and Reuven Lax; O'Reilly, 2018

Components of a big data architecture

Overlap with Lambda architecture



Components of a big data architecture

Overlap with Kappa architecture

For Batch, the pipeline reads from Data storage

